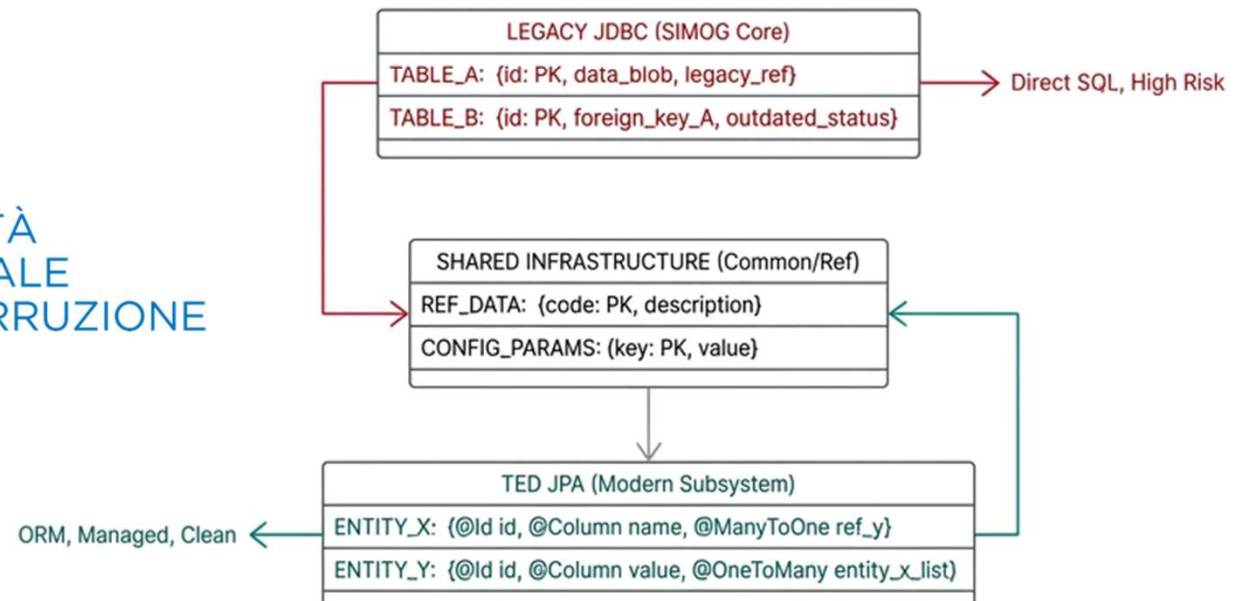


Assessing the SIMOG Data Model and Persistence Architecture

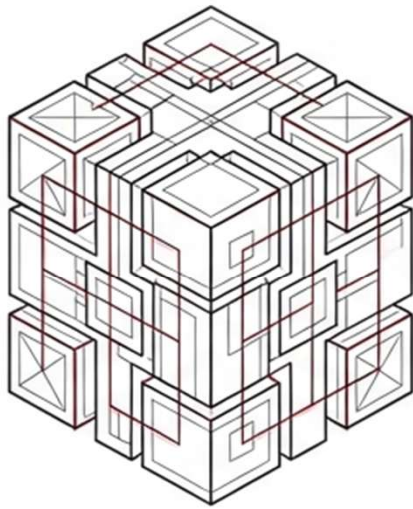
Current state mapping, operational risks, and reverse-engineering priorities for the legacy and TED subsystems.



ANAC AUTORITÀ
NAZIONALE
ANTICORRUZIONE

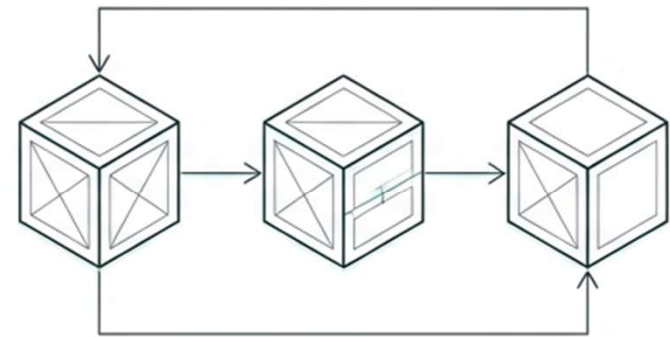


SIMOG operates on a fractured dual persistence architecture



Legacy Core

Relies entirely on direct JDBC and manual SQL execution. Business logic and persistence mechanics are heavily interwoven.



TED Subsystem

Implements a more modern JPA (RESOURCE_LOCAL) model. Operates with explicit entities and a cleaner architectural separation.

Contrasting the core legacy mechanics with the TED persistence model

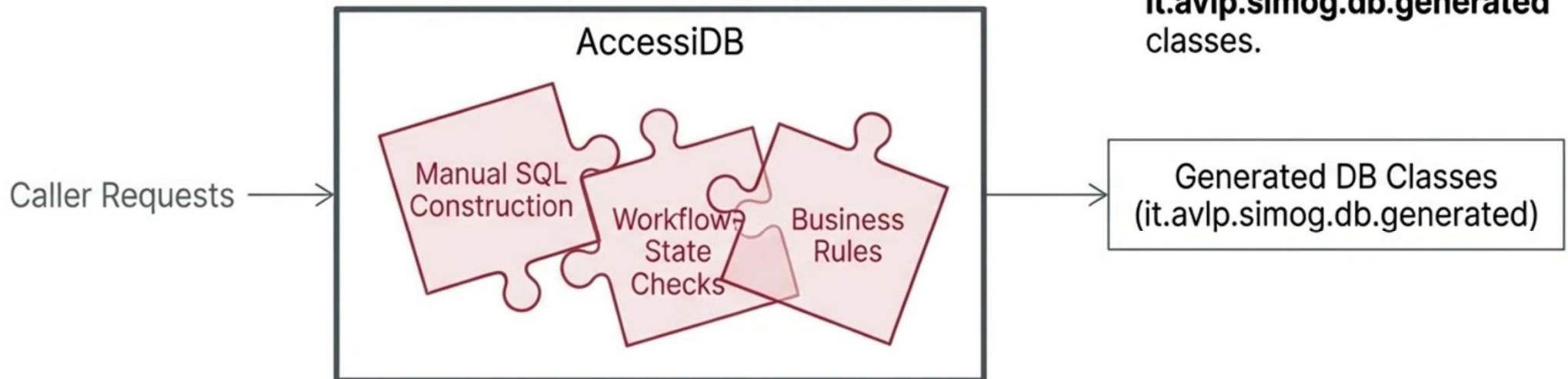
	Core Application	TED Subsystem
Access Technology	Direct JDBC & Manual SQL	JPA (RESOURCE_LOCAL)
Transaction Semantics	Manual, caller-demarcated transactions	Application-driven JPA transactions
Business Logic	Tight coupling (SQL , workflow, and rules interwoven)	Explicit separation via entities
Data Mapping	Partly manual mapping assisted by generated DB classes	Maps directly to declared entities

The legacy core relies on tightly coupled manager classes

The dominant persistence pattern extends the AccessiDB class, forcing callers to manually construct SQL and handle connections.

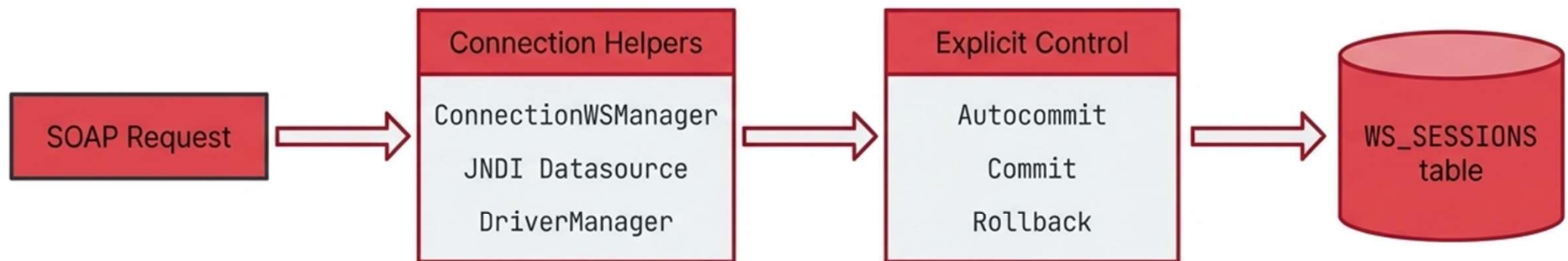
Representative Artifacts:

- GaraManager
- LottoManager
- DocumentoManager
- Result mappings assisted by **it.avlp.simog.db.generated** classes.



SOAP layers utilize dedicated database helpers for session handling

Web Services require specific persistence routing, utilizing ConnectionWSManager and WSSessionManager to open connections and enforce explicit transaction boundaries. WS session and ticket sttte are isolated and persisted in the WS_SESSIONS table.

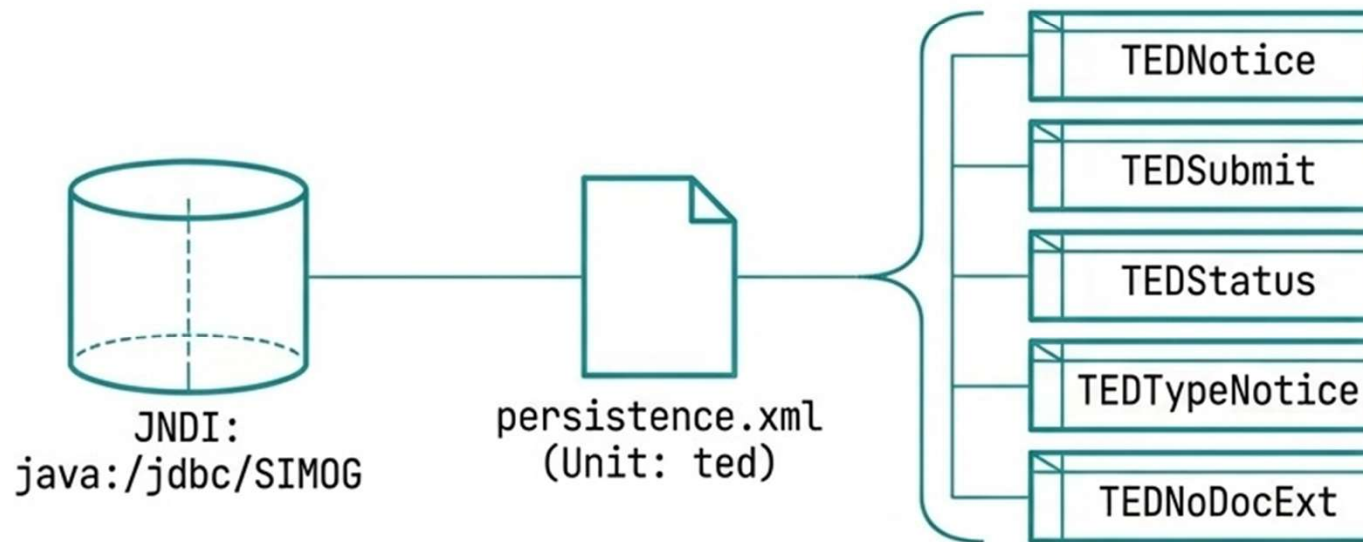


The TED subsystem introduces a cleaner JPA persistence model

Operating via RESOURCE_LOCAL transactions, TED reuses core business concepts (Gara, Lotto, Pubblicazioni) but maps them explicitly.

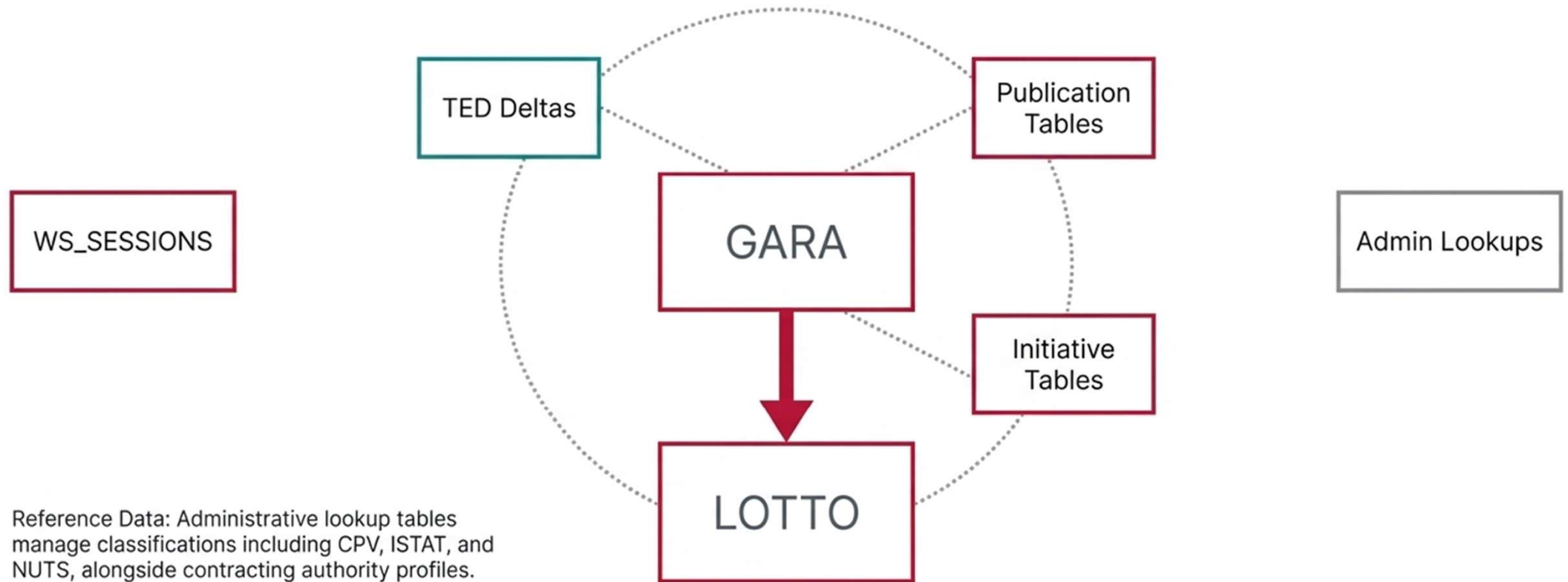
Declared Entities:

- TEDNotice
- TEDSubmit
- TEDStatus
- TEDTypeNotice
- TEDNoDocExt



Reconstructing the persistent concepts from code references

Without strict DDL, entity naming and SQL strings strongly infer a hierarchical core where GARA acts as the parent to LOTTO.



Distinct operational patterns govern data retrieval and modification



Read Patterns

- Ad hoc query execution strictly within manager classes.
- Lookup servlets dedicated to classification data.
- SOAP consultation endpoints for procurement info.
- TED service queries routed through JPA repositories.



Write Patterns

- Servlet and SOAP actions rely on coordinated, single-request transactions.
- Mass-loader utilities execute batch-oriented writes.
- TED flows independently write deltas, notices, and status transitions.

Transaction integrity relies on manual caller discipline



Transactions

The legacy core lacks a visible declarative transaction framework. Servlets and WS endpoints must execute explicit commits and rollbacks, increasing the risk of orphaned data states.



Caching

Caching is strictly tactical and local rather than platform-wide. Evidence shows ServletBase initializing HSqIManager in-memory, but no broad caching strategy exists.

Architectural constraints generate significant operational risk

Coupling

Strong entanglement between SQL and domain logic severely limits code flexibility.

Portability

Database portability is exceptionally low due to manual SQL strings and buried schema logic.

Testing Limits

Unit testing is highly difficult because persistence lacks clean abstraction.

Late Failures

Because validation logic sits dangerously close to persistence execution, operational errors may surface late in the application lifecycle.

Reversing the risk requires targeted reconstruction

To mitigate deep testing constraints and decipher the buried database schema, modernization efforts cannot be broad. Reverse-engineering must surgically target the highest-value legacy junctions where business rules and manual SQL intersect.



Prioritized reverse-engineering roadmap

For a full, actionable data-model reconstruction, incoming teams must target these artifacts in order:

